Information Sciences 580 (2021) 510–523

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

JKT: A joint graph convolutional network based Deep Knowledge Tracing

Xiangyu Song [a], Jianxin Li [a,*], Yifu Tang [a], Taige Zhao [a], Yunliang Chen [b], Ziyu Guan [c]

[a] School of IT, Faculty of Science, Engineering and Built Environment, Deakin University, Geelong, VIC 3220, Australia
[b] China University of Geosciences, China
[c] State Key Lab of ISN, School of Computer Science and Technology, Xidian University, Xi'an, China

ARTICLE INFO

Article history:
Received 8 June 2021
Received in revised form 12 August 2021
Accepted 29 August 2021
Available online 2 September 2021

Keywords:
Knowledge Tracing
Graph neural network
Learning analytics
Intelligent tutoring systems
Network representation learning

ABSTRACT

Knowledge Tracing (KT) aims to trace the student's state of evolutionary mastery for a particular knowledge or concept based on the student's historical learning interactions with the corresponding exercises. Taking the *"exercise-to-concept"* relationships as input, several existing methods have been developed to trace and model students' mastery states. However, these studies face two major shortcomings in KT: 1) they only consider *"exercise-to-concept"* relationships; 2) the multi-hot embeddings lack interpretability. In order to address the above issues, we propose a **J**oint graph convolutional network based deep **K**nowledge **T**racing (**JKT**) framework to model the multi-dimensional relationships of *"exercise-to-exercise"*, and *"concept-to-concept"* into graph and fuse them with *"exercise-to-concept"* relationships. In JKT, it is not only possible to establish connections between exercises under cross-concepts, but also to help capture high-level semantic information and increase the model's interpretability. In addition, sufficient experiments conducted on four real-world datasets have demonstrated that JKT performs better than the other baseline models. We further illustrate a case study to demonstrate its interpretability for learning analysis

© 2021 Elsevier Inc. All rights reserved.

1. Introduction

As a cognitive model with the goal of mastery-based learning, Knowledge Tracing (KT) is first introduced into the field of intelligent tutoring systems in [6]. It seeks to monitor the evolving state of students' mastery of a specific learning concept or knowledge when they interact with several corresponding exercises. The intent of this task is to personalise the sequence of practice to enable efficient mastery of the concepts, and to accurately predict student performance. With the rise of the internet and online education, knowledge tracing is being used more and more widely on various educational platforms, and is becoming a popular research topic.

Various approaches have been proposed to solve the KT problems. There are two most representative models: Bayesian Knowledge Tracking (BKT) and Deep Knowledge Tracing (DKT). BKT [6] models a student's potential knowledge state as a set of binary variables, each representing whether the student understands the corresponding concept.[26] applies RNNs [28] to the field of knowledge tracing, denoted as DKT. Compared to BKT, DKT takes the advantage of multiple free parameters and

* Corresponding author.
E-mail addresses: xiangyu.song@deakin.edu.au (X. Song), jianxin.li@deakin.edu.au (J. Li), tangyif@deakin.edu.au (Y. Tang), chris.zhao@research.deakin.edu.au (T. Zhao), cyl_king@hotmail.com (Y. Chen), zyguan@xidian.edu.cn (Z. Guan).

https://doi.org/10.1016/j.ins.2021.08.100
0020-0255/© 2021 Elsevier Inc. All rights reserved.

cross-input of multiple concepts, resulting in better results. Following the mainstream approach of DKT, there are many other developed methods. Inspired by memory-enhanced neural networks, dynamic Key-Value Memory Networks (DKVMN) [41] creates a static matrix as well as a dynamic matrix to store all the concepts and maintain the student's knowledge learning state, respectively. SAKT [24] uses self-attentive mechanisms [33] to assign weights to previously answered exercises in order to predict student performance on the specific exercises. EERNN [31] uses the textual content of exercises and students' practice records to predict student performance. Even though all these models can make good use of the explicit relationship between exercises and concepts to predict the student's mastery state of a certain concept, such models often fail to make accurate prediction when a student is confronted with a concept that has not been previously covered, and are also hard to capture the high-level semantic information.

To fill in these gaps, we propose a Joint graph convolutional network based deep Knowledge Tracing (JKT) framework that focuses on improving prediction accuracy as well as the interpretability of the method by exploring the implicit relationships of "*exercise-to-exercise*" and "*concept-to-concept*". As shown in Fig. 1, We first take one student's historical interactions as a sample and count all the co-occurred exercise pairs that he answers correctly. We then consider a student to have mastered a concept when he obtains an above-average accuracy rate for that concept. Similarly, we count all co-occurring pairs of concepts that he has mastered and then calculate the ratio of the number of these co-occurring concepts that he has mastered to the number of occurrences of these co-occurring concepts, indicating the co-mastery rate.

We can then establish the exercise influence sub-graph and the concept influence sub-graph based on the accumulated co-correctness rate of each pair of co-occurred exercises and co-mastery rate of each pair of co-occurred concepts, respectively. Given the established sub-graphs, we can learn the embeddings of each exercise and its corresponding concept through a two-layer Graph convolutional network. Finally, we input these dense embeddings instead of traditional one-hot or multi-hot vector representations into a RNN-based prediction layer to make prediction.

Compared to the existing methods, our proposed graph structure representation and the exploration of the relationships of "*exercise-to-exercise*" and "*concept-to-concept*" allow for better and multi-dimensional establishment of connections between exercises under different concepts. Furthermore, with powerful graph embeddings and graph convolution, we can transform long and sparse one-hot or multi-hot vector representations into dense ones. At the same time, due to involving the co-correctness rate and co-mastery rate, the embeddings with higher co-correctness rate and co-mastery rate are more likely to have higher similarity in the vector expressions. This feature can help to improve the prediction accuracy as well as the interpretability of the method.

The contributions of our work are summarized as follows:

- As far as we know, this is the first attempt to study the "*exercise-to-exercise*", and "*concept-to-concept*" relationships in the joint attributed KT graph structure. Our work can learn the reliable connections between exercises under different concepts, further promoting the accuracy of student performance prediction and enhancing the interpretability of KT-based applications.
- We propose a novel framework called Joint graph convolutional network deep Knowledge Tracing (JKT), which includes both the exercise influence sub-graph and the concept influence sub-graph. Benefiting from this multidimensional sub-graph fusion, the proposed JKT can take into account the importance of all "*exercise-to-concept*", "*exercise-to-exercise*" and "*concept-to-concept*" relationships. Furthermore, another merit of JKT is able to capture the semantic information of students' learning states at higher levels in the knowledge tracing process.
- Sufficient experiments and the corresponding ablation study on four real-world datasets have been conducted to demonstrate that JKT perform better than other baseline models. More importantly, through an in-depth analysis of the "*exercise-to-exercise*", and "*concept-to-concept*" relationships, the proposed JKT demonstrates its strong interpretability in learning analysis.

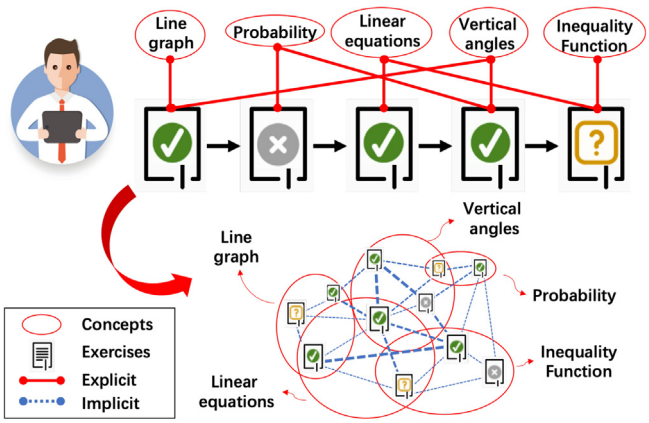

Fig. 1. An illustrative example of the transformation from the original sequential representation to a multi-dimensional graphical representation.

The rest of the paper is organised as follows. In Section 2, we review some related work on knowledge tracing, graph embeddings and graph convolutional networks. Section 3 elaborates on the problem definition of knowledge tracing and the definitions of two sub-graphs, the Exercise influence sub-graph and the Concept influence sub-graph. Section 4 presents the framework of the JKT model and a detailed theoretical procedure. Section 5 shows the experimental evaluation and discussion. We conclude the paper and present the future work in Section 6.

2. Related work

In this section, we briefly review the related work from three aspects, knowledge tracing, network Embedding and graph representation learning.

2.1. Knowledge Tracing

Knowledge Tracing is to model students' learning states based on the sequences of students' learning activities. In the field of Intelligent Tutoring Systems (ITS) [2], knowledge tracing is one of the most challenging tasks to solve because these systems need to accurately predict students' performance and capture a student's mastery of knowledge.

Bayesian Knowledge Tracing (BKT) [6] is widely used as a learning state modelling method in ITS. It adopts Hidden Markov model (HMM) [3] to maintain the binary variants which represents whether the students understand the corresponding concepts. BKT solves the parameter tuning problem using the EM algorithm [8] and the brute force algorithm [22].

Deep Knowledge Tracing (DKT) [26], a pioneering algorithm for modelling students' learning states using recurrent neural networks, has reported substantial improvements in its predictive performance. [21] compares the differences between DKT and BKT and find out that the key advantage of DKT is that it is a neural network model with multiple free parameters, whereas BKT is a probabilistic model with limited free parameters.

GKT [23] builds a graph of concept relations and learns them explicitly. However, this approach uses only concepts as input, which ignore the influence between exercise and exercise. Some deep KT methods take exercise characteristices into account for predictions. Dynamic Student Classification on Memory Networks (DSCMN) [20] utilizes exercise difficulty to help distinguish the exercises related to the same concepts. DHKT[36] augments DKT by using the relations between exercises and concepts to get exercise representations, which, however, fails to capture the "*exercise-to-exercise*" and "*concept-to-concept*" relationships. In this paper we design a Joint Graph Convolutional Network to extract the deep implicit information hidden in the "*exercise-to-concept*" graph.

To deal with the problem of long-term dependencies in knowledge tracing, [1] proposed a model called Sequential Key-Value Memory Networks (SKVMN). It applies the improved LSTM with hop to improve the ability of the exercises' sequential long-term dependencies extracting. EERNNA [31] weights and aggregates all historical student states for correlations between current exercises and historical problems. EKT [19] is an extension of the EERNN model, which also tracks students' knowledge acquisition of multiple concepts. Specifically, it models the relationships between the underlying knowledge concepts to augment the EERNN model. SAINT+ [30] is an extension of SAINT, the first transformer-based knowledge tracing model with an encoder-decoder structure. A sequence of exercise embeddings is fed into the encoder, while the decoder can get the output of the exercise and its corresponding response embedding to compute the final output. And AKT [9] combines a flexible attention-based neural network model with a range of new, interpretable model components inspired by cognitive and psychometric models allowing learners' future responses to assessment exercises to be related to their past responses. Our approach differs from these works in that they aggregate related hidden states into a new state for prediction, whereas we first mine the deep "*exercise-to-exercise*", and "*concept-to-concept*" information, and then predictions are made based on the historical interaction information of the students.

2.2. Network embedding

Existing network embedding algorithms can be divided into two categories: those [5,10,25,32,35,40,37,18] are used for ordinary networks, using only topological information for embedding, while those [11,13,16,17,38] are used for non-ordinary networks, as in the case of attribution networks, not only topological information but also auxiliary information, such as content information/attributes of nodes, should be considered when embedding is applied.

To embed general networks, there are some methods to learn embedding using random walk or edge sampling, such as [25], node2vec [10] and LINE [32]. These methods learn and incorporate the Skip-Gram neural network architecture with negative sampling originally proposed for word embeddings and allow implicit approximation learning through matrix factorization [27]. Wang et al. [35] proposed the SDNE model to extract the highly nonlinear graph structure information while keeping the global and local connection information of the graph. And it also incorporates the community structure of the network into the resultant embedding, preserving both the micro and community structure. There are also several works to obtain embeddings of non-plain networks, where rich information such as labels, node attributes and text content are added to the original topological network structure, such as [13,17,12]. Huang et al. [13] used graph Laplacian techniques in order to learn joint embeddings from topologies and attributes. Kipf and Welling [16] proposed a graph convolutional neural network model based on attribute networks and fused it with a variational autoencoder architecture. Hamilton et al. [11]

proposed a model to learn node representations by sampling and aggregating features of a node's local neighborhood, named GraphSAGE. Zhang et al. [41] and Gao and Huang [13] presented ANRL and DANE, using their customed deep neural network architecture to learn node embeddings while capturing potentially high nonlinearities in topology and properties. Their results show that capturing topological features combined with different types of auxiliary information can make the embedding of nodes richer and can be applied to many real-world networks. However, these models simply learn the embedding of the nodes without noticing the uncertainty in the embedding process. Based on these works, this paper proposes a novel joint embedding model for knowledge tracing that can be used to obtain high levels of information and improve student performance prediction accuracy. By constructing and learning "*exercise-to-exercise*" and "*concept-to-concept*" relations, high levels of information can be obtained and the accuracy of student performance prediction can be improved.

2.3. Graph convolutional networks

Recently, there is a growing interest in the application of convolutional neural networks to graph-structured data. The convolution procedure in the Fourier domain is described by the computation of the decomposition of the Laplacian graph itself [4]. To reduce the difficulty of the convolution, [16] proposes the GCN model, which simplifies the previous solution by first-order approximation [39] to the local spectral filter [34] on the graph. Specifically, the convolution operation is the aggregation of all neighbouring feature vectors for each node, which is then put into a fully connected layer and a nonlinear activation function for transformation. However, an important problem in their proposed model is that the representations of the central and neighbouring nodes are aggregated together with the same weights, which makes them untrainable. [11] recently suggests GraphSage to further expand the GCN process in an inductive way. This technique is used to sample the fixed-size neighbourhood of each node to avoid too many changes in the entire Laplacian graph. They also improve GCN by associating the representation of each node with an aggregated neighbourhood vector to learn interaction weights. While considerable progress has been made, previous GCN analysis work focuses primarily on a single graph and is never used in the area of knowledge tracing. In this work, we propose a novel framework that extends GCN to the KT domain and fuses two sub-graphs.

In this section, we elaborate on the problem definition of knowledge tracing and the definition of the Exercise influence sub-graph and the Concept influence sub-graph. Fig. 2 clearly illustrates the differences and connections between the three relationships. The specific definitions are given as follows.

Problem 1 (*Knowledge Tracing*). Given a sequence of the interaction observations taken by a student $s \in \mathcal{S}, s = \{\mathcal{X}_0 \dots \mathcal{X}_T\} = \{(e_0, a_0), (e_1, a_1), \dots, (e_T, a_T)\}$ on a specific concept $c \in \mathcal{V}_c$, where $\mathcal{X}_t = \{e_t, a_t\}$ for the exercise $e \in \mathcal{V}_e$ being answered at the time step $t \in T$ with whether or not the exercise was answered correctly $a_t \in \{0, 1\}$. The knowledge tracing task is to predict the next interaction $\mathcal{X}_{t+1}$.

To address the problem, we reconstruct the entire input data and explore the graph structure relationships between the exercises and concepts. We construct the "*exercise-to-exercise*"-based exercise influence sub-graph and "*concept-to-concept*"-based concept influence sub-graph, after which we fuse them into a Joint attributed knowledge tracing graph.

The exercise influence sub-graph derives from students' transitions between exercises. Its establishment not only improves the previous model's inability to distinguish the exercises on the same concept, but also provides a wealth of "*exercise-to-exercise*" graph structured information.

Definition 1 (*Exercise influence sub-graph*). Given an exercise set $\mathcal{V}_e$ and an "exercise-to-exercise" interaction set $\mathcal{E}_e$, an exercise influence sub-graph is a graph $\mathcal{G}_e = (\mathcal{V}_e, \mathcal{E}_e)$ that takes the exercises in $\mathcal{V}_e$ as the vertices and the interactions in $\mathcal{E}_e$ as the edges. The weighted influence Q_e^{ij} $(i, j \in \mathbb{N}, i, j \leqslant m)$ of the edges of co-ocurred exercises $\mathcal{V}_e^i$ and $\mathcal{V}_e^j$ is measured by

$$Q_e^{ij} = \frac{f^c(\mathcal{V}_e^i, \mathcal{V}_e^j)}{\sum_m f^o(\mathcal{V}_e^i, \mathcal{V}_e^m)} \tag{1}$$

where Q_e^{ij} is the co-correctness rate of exercises $\mathcal{V}_e^j$ and $\mathcal{V}_e^i$ among all the answered co-occurred exercises involving $\mathcal{V}_e^i$. f^c and f^o denote the count of co-correctness and co-occurrence respectively. Edge $\mathcal{E}_e$ $(\mathcal{V}_e^j, \mathcal{V}_e^i)$ only exists when Q_e^{ij} is larger than a certain threshold τ.

Example 1. As shown in Fig. 2(b), we construct an example of the exercise influence sub-graph to model one KT database. It consists of four different concepts and "*exercise-to-exercise*" between exercises on the same concept.

During the process of KT, existing models often ignore the connections between different concepts, resulting in such models being unable to make good predictions when students are confronted with a concept that has not been previously covered. The concept influence sub-graph can fill this gap by bridging different concepts and quantifying the influence of each "*concept-to-concept*" relationship.

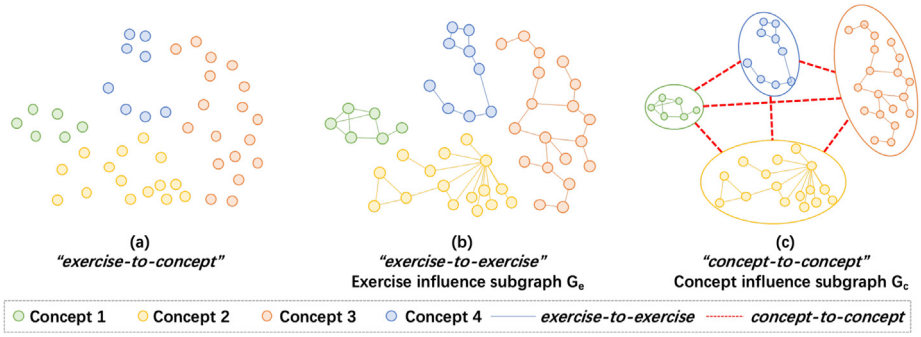

Fig. 2. The multi-dimensional relationships of *"exercise-to-concept"*, *"exercise-to-exercise"*, and *"concept-to-concept"*.

Definition 2 (*Concept influence sub-graph*). Given a concept set $\mathcal{V}_c$ and a "concept-to-concept" interaction set $\mathcal{E}_c$, a concept influence sub-graph is a graph $\mathcal{G}_c = \left(\mathcal{V}_c, \mathcal{E}_c, Q_c^{ij}\right), i,j \in \mathbb{N}, (i,j \leqslant n)$ that takes the concepts in $\mathcal{V}_c$ as the vertices and the interactions in $\mathcal{E}_c$ as the edges. The weighted influence Q_c^{ij} of the edges of co-ocurred concepts $\mathcal{V}_c^i$ and $\mathcal{V}_c^j$ is measured by

$$Q_c^{ij} = \frac{f^m\left(\mathcal{V}_c^i, \mathcal{V}_c^j\right)}{\sum_n f^o\left(\mathcal{V}_c^i, \mathcal{V}_c^n\right)} \tag{2}$$

where Q_c^{ij} is the co-mastery rate (mastery: a student gets the above-average correctness of the exercises on the concept) of concepts $\mathcal{V}_c^j$ and $\mathcal{V}_c^i$ among all the engaged co-occurred concepts involving $\mathcal{V}_c^i$. f^m and f^o denote the count of co-mastery and co-occurrence of the concepts respectively. Edges $\mathcal{E}_c$ $(\mathcal{V}_c^j, \mathcal{V}_c^i)$ exists on each co-ocurred concept and Q_c^{ij} is defined as the weight on the edge $\mathcal{E}_c$ $(\mathcal{V}_c^j, \mathcal{V}_c^i)$.

Example 2. As shown in Fig. 2(c), four different concepts can be connected via *"concept-to-concept"* relationship. Different *"concept-to-concept"* relationships always reveal different semantic information. For example, the weight on the edge between *"Linear equations"* and *"Quadratic equation"* is probably larger than the one between *"Linear equations"* and *"Vetical angles"*. This is because both *"Linear equations"* and *"Quadratic equation"* examine your algebraic capability, while *"Vetical angles"* examine your geometric capability.

After applying graph convolutional network (GCN) technique on the exercise influence sub-graph and concept influence sub-graph, we can obtain two low-dimensional vectors to represent each exercise together. They each contain the relationships between exercises under the same concept and the corresponding concept. These vectors can be used as attributes for each exercise after concatenation. These low-dimensional and dense attributes contain structured information of *"exercise-to-concept"*, *"exercise-to-exercise"* and *"concept-to-concept"* relationships. Finally, we construct a bipartite graph of these exercises with attributes and students called the joint attributed knowledge tracing graph.

Definition 3 (*Joint attributed knowledge tracing graph*). Given a student set $\mathcal{S}$, an exercise set $\mathcal{V}_e$, a "student-exercise" interaction set $\mathcal{E}_s$, and the attribute $\mathcal{A}_e$ associated to exercises, the joint attributed knowledge tracing graph is an attributed undirected bipartite graph $\mathcal{G} = (\mathcal{S}, \mathcal{V}_e, \mathcal{E}_s, \mathcal{A}_e)$ that takes the students in $\mathcal{S}$, exercises in $\mathcal{V}_e$ as the vertices and the interactions in $\mathcal{E}_s$ as the edges.

Example 3. As shown in Fig. 3, students as samples can be connected with the exercise set $\mathcal{V}_e$ via a "student-exercise" interaction set $\mathcal{E}_s$. The structure of this graph reveals the relationship between students with embedded exercises. For example, the fact that student $\mathcal{S}_4$ got exercises $\mathcal{V}_e^1$ and $\mathcal{V}_e^3$ correct not only updated student $\mathcal{S}_4$'s state of mastery of the concepts corresponding to exercises $\mathcal{V}_e^1$ and $\mathcal{V}_e^3$, but also updated the student's state of mastery of all other concepts since the embedded representations of these two exercises included their relationships to other exercises under the same concept and the relationship of its concept to other concepts, thus allowing a basic prediction probability of the upcoming exercises $\mathcal{V}_e^2, \mathcal{V}_e^4$ and $\mathcal{V}_e^5$ for student $\mathcal{S}_4$.

3. Method

According to the problem defined in Section 3, we propose **JKT**, a joint graph convolutional network based deep knowledge tracing framework, to obtain embeddings of graph structured *"exercise-to-exercise"* and *"concept-to-concept"* relationships. Fig. 3 presents an outline of our proposed framework. In the following subsections, we will explain the JKT

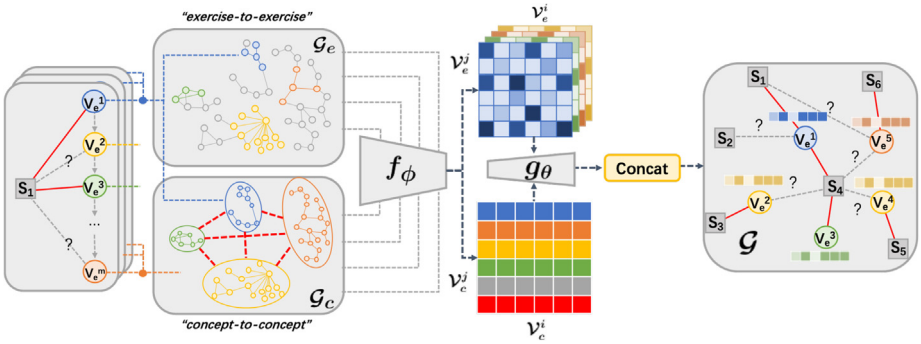

Fig. 3. An overview of JKT framework. The structural components of our proposed framework are inspired from VAE and DKT. The structure of JKT is shown in the Fig. 3 and consists of three main parts: subgraph construction, "inference model f_ϕ – generative model g_θ" model and predictive model. Subgraph construction is achieved by learning the multi-dimensional relationships of "exercise-to-exercise", and "concept-to-concept" from the raw data and converting them into graph form. Then we learn the high-level representations of the exercises and concepts separately according to the process structure "inference model f_ϕ – generative model g_θ" and combine them in the same space. Finally, the vector of high-level dense representations for each exercise is fed into the prediction model as an attribute of each exercise, where the prediction model follows the structure of DKT. Ultimately, the inference, generation and prediction models will be jointly optimized.

framework from the following aspects: 1) The intuition, 2) Joint GCN based graph embeddings, and 3) Attributed knowledge tracing graph generation and 4) The loss functions. All the main notations can be found in Table 4.

3.1. The intuition

The key idea of JKT is to mine deeper data relationships from existing data, (i.e. "*exercise-to-exercise*" and "*concept-to-concept*") and represent them in a sensible way. Through the construction of $\mathcal{G}_c$ and $\mathcal{G}_e$, we have succeeded in representing these relationships as graph structures which can better uncover the relationships between different exercises and concepts. We then propose a two-layer model for the reconstruction of $\mathcal{G}_c$ and $\mathcal{G}_e$, which consists of an inference model f_ϕ with trainable parameters ϕ and a generative model g_θ with trainable parameters θ, respectively. By doing so, we can encode the features of nodes and attributes as two Gaussian distributions and effectively reconstruct both real and binary weighted edges and attributes. Finally, we replace the traditional multi- hot embeddings by the learned dense attributes for each exercise. The attributes consist of the concatenated exercises and concepts embeddings with the structural information of embedding "*exercise-to-exercise*" and "*concept-to-concept*" relationships. At the same time, the newly learned attributes will be interpretable to a certain extent, which will be illustrated in Section 5.

3.2. Joint GCN based graph embeddings

In this subsection, we will go through the first layer of the inference model f_ϕ to learn the embedding of the two subgraphs and try to retain as much structural information as possible in the process. Given the established sub-graphs $\mathcal{G}_e$ and $\mathcal{G}_c$, We first need to learn two mapping functions Ξ_c and Ξ_e in an unsupervised way according to the following conditions:

$$\begin{aligned} \mathcal{G}_c &\xrightarrow{\Xi_c} \mathbf{Z}_c^{\mathcal{V}}, \mathbf{Z}_c^{\mathcal{A}} \\ \mathcal{G}_e &\xrightarrow{\Xi_e} \mathbf{Z}_e^{\mathcal{V}}, \mathbf{Z}_e^{\mathcal{A}} \end{aligned} \tag{3}$$

By doing this, the structure of the network and some hidden high-level semantic information can be maximally preserved by $\mathbf{Z}^{\bar{\mathcal{V}}} \in \mathbb{R}^{N\times D}$ and $\mathbf{Z}^{\mathcal{A}} \in \mathbb{R}^{F\times D}$, respectively, with $Z_i^{\mathcal{V}} \in Z^{\mathcal{V}}, Z_j^{\mathcal{A}} \in \mathbf{Z}^{\mathcal{A}}, N, F$ and D are inferred as the number of nodes and attributes, respectively, and the dimension size of the embedding for the corresponding node and attribute.

Inspired by [14], two neural network models, an inference model f_ϕ with trainable parameters ϕ and a generative model g_θ with trainable parameters θ will be applied as the probabilistic encoder and probabilistic decoder, respectively. The gradient ascent learning method [42] will be proformed on all model parameters. Table 1.

To encode from the two subgraphs to the embedding, we superimposed a parallelled GCN and its corresponding fully connected layer, and they consist of a nonlinear mapping function [7] to map the adjacency matrix $\mathbf{A}_e$ and $\mathbf{A}_c$ as well as attribute matrix $\mathbf{X}_e$ and $\mathbf{X}_c$ of sub-graphs $\mathcal{G}_e$ and $\mathcal{G}_c$. We apply GCN [4] rather than other neural network models for node inference because it produces the best performance. We define the two-layer GCN as:

Table 1
Notations in the paper.

Symbol	Description
$\mathcal{G}$	a joint attributed KT graph
$\mathcal{G}_e$	an exercise influence sub-graph
$\mathcal{G}_c$	a concept influence sub-graph
$\mathcal{V}_e$, $\mathcal{V}_c$	set of exercises(e), concept(c) nodes
$\mathcal{S}$	set of student(s) nodes
$\mathcal{E}_s$	a "student-exercise" interaction set
$\mathcal{A}^e$, $\mathcal{A}^c$	set of exercise, concept attributes
$N_c = \lvert\mathcal{V}_c\rvert$, $N_e = \lvert\mathcal{V}_e\rvert$	number of exercise, concept nodes
$F_c = \lvert\mathcal{A}_c\rvert$, $F_e = \lvert\mathcal{A}_e\rvert$	number of exercise, concept attributes
$\mathbf{A}_c \in \mathbb{R}^{N_c\times N_c}$, $\mathbf{A}_e \in \mathbb{R}^{N_e\times N_e}$	adjacency matrix of exercise, concept nodes
$\mathbf{X}_c \in \mathbb{R}^{N_c\times F_c}$, $\mathbf{X}_e \in \mathbb{R}^{N_e\times F_e}$	attribute information matrix
$\mathbf{Z}^{\mathcal{V}_c} \in \mathbb{R}^{N_c\times D_c}$, $\mathbf{Z}^{\mathcal{V}_e} \in \mathbb{R}^{N_e\times D_e}$	latent representation matrix
$\mathbf{Z}^{\mathcal{A}_c} \in \mathbb{R}^{F_c\times D_c}$, $\mathbf{Z}^{\mathcal{A}_e} \in \mathbb{R}^{F_e\times D_e}$	latent representation matrix

$$\begin{aligned}\mathbf{H}_{\mathcal{V}_{c,e}} &= \mathrm{ReLU}\left(\tilde{\mathbf{A}}\tilde{X}\mathbf{W}_{\mathcal{V}_{c,e}}\right)\\ \left[\boldsymbol{\mu}_{\mathcal{V}_{c,e}}, \boldsymbol{\sigma}_{\mathcal{V}_{c,e}}\right] &= \tilde{\mathbf{A}}\mathbf{H}_{\mathcal{V}_{c,e}}\mathbf{W}_{\mathcal{V}_{c,e}}\end{aligned} \tag{4}$$

where $\boldsymbol{\mu}_{\mathcal{V}_{c,e}}$ and $\boldsymbol{\sigma}_{\mathcal{V}}^{c,e}$ are the means and variances of the learned embeddings of nodes $\mathcal{V}_e$ and $\mathcal{V}_c$, $\mathrm{ReLU}(\cdot)$ has been chosen as the activation function here because it does not suffer from gradient saturation and gradient disappearance on the $x > 0$ region, $\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}$ is the symmetrically normalized adjacency matrix.

Next, we define the fully connected layer for embedding properties:

$$\begin{aligned}\mathbf{H}_{\mathcal{A}_{c,e}} &= \tanh\left(\mathbf{X}^{\mathrm{T}}\mathbf{W}_{\mathcal{A}_{c,e}} + \mathbf{b}\right)\\ \left[\boldsymbol{\mu}_{\mathcal{A}_{c,e}}, \boldsymbol{\sigma}_{\mathcal{A}_{c,e}}\right] &= \mathbf{H}_{\mathcal{A}_{c,e}}\mathbf{W}_{\mathcal{A}_{c,e}} + \mathbf{b}\end{aligned} \tag{5}$$

Here we use the method of taking the mean $\boldsymbol{\mu}_{\mathcal{A}}$ and variance $\boldsymbol{\sigma}_{\mathcal{A}}$ to learn the embeddings of exercise $\mathcal{V}_e$ and concept $\mathcal{V}_c$, $\mathbf{b}$ here is the bias, The $\tanh(\cdot)$ function is 0-mean, so here we use it as the activation function for this layer and $\phi_2 = [\mathbf{W}_{\mathcal{A}}, \mathbf{W}_{\mathcal{A}}, \mathbf{b}, \mathbf{b}]$ are the parameters that need to be trained in this layer.

3.3. Attributed KT graph generation

In this subsection, we will use the second layer of the generative model g_θ to reconstruct the Joint attributed knowledge tracing graph with the learned embeddings and to restore as much of the original structural information as possible. After obtaining the mean and variance of all embeddings of the exercise $\mathcal{V}_e$ and the concept $\mathcal{V}_c$. We adopt the reparameterization trick [15] to make the transformation into deterministic variables $\mathbf{Z}^{\mathcal{V}_e}$ and $\mathbf{Z}^{\mathcal{V}_c}$ that are able to propagate the inference model and the generative model gradient.

The generative model g_θ's objective is to decode the latent embeddings $\mathbf{Z}^{\mathcal{V}_e}$ and $\mathbf{Z}^{\mathcal{V}_c}$ to generate random variables and reconstruct the original graph structure with connections and information. To be detailed, we compute $\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{V}}(i,j)}$ and $\boldsymbol{\sigma}^2_{\mathcal{E}_{\mathcal{V}}(i,j)}$ by the following equation:

$$\left[\mu_{\mathcal{E}_v(i,j)}, \sigma^2_{\mathcal{E}_v(i,j)}\right] = g_{\theta_1}\left(\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_j^{\mathcal{V}}\right) \tag{6}$$

where g_{θ_1} is a generative model for reconstructing edges. Here, we refer to $p_{\theta_1}\left(\mathbf{A}_{ij}|\mathbf{Z}_i^{\mathcal{V}_e}, \mathbf{Z}_j^{\mathcal{V}_c}\right)$ and $p_{\theta_2}\left(\mathbf{X}_{ia}|\mathbf{Z}_i^{\mathcal{V}_e}, \mathbf{Z}_a^{\mathcal{V}_c}\right)$ as the probabilistic decoders, since given the latent embeddings of exercise $\mathbf{Z}^{\mathcal{V}_e}$ and concept $\mathbf{Z}^{\mathcal{V}_c}$, this produces a distribution of possible corresponding values for the observed edges and attributes. Then the edges can be reconstructed as follows:

(a) For "*concept-to-concept*" edges $\mathcal{E}_c$,

$$p_{\theta_1}\left(\mathbf{A}_{ij}|\mathbf{Z}_i^c, \mathbf{Z}_j^c\right) = \mathcal{N}\left(\boldsymbol{\mu}_{\mathcal{E}_c(i,j)}, \boldsymbol{\sigma}^2_{\mathcal{E}_c(i,j)}\mathbf{I}\right) \tag{7}$$

(b) For "*exercise-to-exercise*" edges $\mathcal{E}_e$,

$$p_{\theta_1}\left(\mathbf{A}_{ij}|\mathbf{Z}_i^e, \mathbf{Z}_j^e\right) = \mathrm{Ber}\left(\boldsymbol{\mu}_{\mathcal{E}_e(i,j)}\right) \tag{8}$$

On the other hand, given embeddings of exercise $\mathcal{V}_e$ and concept $\mathcal{V}_c$ and the corresponding attributes, we compute $\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{A}}(i,j)}$ and $\boldsymbol{\sigma}^2_{\mathcal{E}_{\mathcal{A}}(i,j)}$ by:

$$\left[\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{H}}(i,j)}, \boldsymbol{\sigma}^2_{\mathcal{E}_{\mathcal{A}}(i,j)}\right] = g_{\theta_2}\left(\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_a^{\mathcal{A}}\right) \tag{9}$$

where g_{θ_2} is a generative model for reconstructing attributes. Then the attribute $(i,j) \in \mathcal{E}_{\mathcal{A}}$ generation process can be summarized as follows:

(a) For "*concept-to-concept*" attributes,

$$p_{\theta_2}\left(\mathbf{X}_{ia}|\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_a^{\mathcal{A}}\right) = \mathcal{N}\left(\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{H}}(i,j)}, \sigma^2_{\mathcal{E}_{\mathcal{A}}(i,j)}\mathbf{I}\right) \tag{10}$$

(b) For "*exercise-to-exercise*" attributes,

$$p_{\theta_2}\left(\mathbf{X}_{ia}|\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_a^{\mathcal{A}}\right) = \mathrm{Ber}\left(\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{H}}(i,j)}\right) \tag{11}$$

In the above equations $\theta = [\theta_1, \theta_2]$ are packed as the whole parameters to be trained in the inference model. Since all the original edges and attributes in our experimental datasets are binary, we implement the model g by means of inner products between latent variables, i.e.

$$\begin{aligned} g_{\theta_1}\left(\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_j^{\mathcal{V}}\right) &= \mathrm{sigmoid}\left(\mathbf{Z}_i^{\mathcal{V}T}\mathbf{Z}_j^{\mathcal{V}}\right) \\ g_{\theta_2}\left(\mathbf{Z}_i^{\mathcal{V}}, \mathbf{Z}_a^{\mathcal{A}}\right) &= \mathrm{sigmoid}\left(\mathbf{Z}_i^{\mathcal{V}T}\mathbf{Z}_a^{\mathcal{A}}\right) \end{aligned} \tag{12}$$

sigmoid$(\cdot)$ is used as the activation function here. After the process of embedding, we need to concat the embeddings of $\mathbf{Z}^{\mathcal{V}}, \mathbf{Z}^{\mathcal{A}}$ and assign them to the corresponding exercise $\mathcal{V}_e$ and form the joint attributed knowledge tracing graph is an attributed undirected bipartite graph $\mathcal{G} = (\mathcal{S}, \mathcal{V}_e, \mathcal{E}_s, \mathcal{A}_e)$, where the students in $\mathcal{S}$, exercises in $\mathcal{V}_e$ are the vertices and the interactions in $\mathcal{E}_s$ are the edges and $\mathcal{A}_e$ denote the learned concatenations of the embeddings.

3.4. Loss function

Finally, to make predictions about student performance in answering the questions, we train the inference model f_ϕ and the generative model g_θ to update the parameters under the constraints as follows:

$$\begin{aligned} \hat{r}^e_{ij} &= \sigma\left(e_i^T e_j\right), i,j \in [1,\ldots,|e|], \\ \hat{r}^c_{ij} &= \sigma\left(c_i^T c_j\right), i,j \in [1,\ldots,|c|]. \end{aligned} \tag{13}$$

We minimize the cross entropy to make vertex features preserve the implicit relations:

$$\begin{aligned} \mathcal{L}_e &= \sum_{i=1}^{|e|}\sum_{j=1}^{|e|} -\left(r^e_{ij}\log\hat{r}^e_{ij} + \left(1 - r^e_{ij}\right)\log\left(1 - \hat{r}^e_{ij}\right)\right) \\ \mathcal{L}_c &= \sum_{i=1}^{|c|}\sum_{j=1}^{|c|} -\left(r^c_{ij}\log\hat{r}^c_{ij} + \left(1 - r^c_{ij}\right)\log\left(1 - \hat{r}^c_{ij}\right)\right) \end{aligned} \tag{14}$$

The loss function of JKT is degined according to the negative log likelihood. We sequentially input the sequence of students' exercise embedding from step 1 to T. Specifically, at t-th step, $\widetilde{a}_t$ is the predicted value of exercise e_t through JKT framework, a_t is the true value, thus the overall loss can be defined as:

$$\mathcal{L}_p = -\sum_{t=1}^{T}\left(a_t \log\tilde{a}_t + (1 - a_t)\log\left(1 - \widetilde{a}_t\right)\right) \tag{15}$$

By synthesizing the above objectives, the joint loss function of JKT is formulated as follows, it is minimized by the Adam optimization [14].

$$\mathcal{L} = \min\ \alpha_1(\mathcal{L}_e + \mathcal{L}_c) + \alpha_2\mathcal{L}_p \tag{16}$$

where $\alpha_1, \alpha_2 > 0$ are two independent hyper-parameters. The objective of JKT is to handle the multi-dimensional relationships of "*exercise-to-exercise*", and "*concept-to-concept*", resulting in obtaining the good high-level semantic information. Eq. (14) minimizes the reconstruction loss and the KL loss to learn high-level subspaces and make the latent representations for both exercise and concept embeddings respectively. In addition, the inference model f_ϕ and generative model g_θ will get a deeper semantic understanding and preserve the original structures. Hence, by solving the overall optimization problem in Eq. (16), we can achieve the goal of JKT.

3.5. The JKT Algorithm

To make this framework better understood, Algorithm 1 summarizes the major steps of the whole process. The main idea of our algorithm is to learn latent representation matrix for the exercise $\mathbf{Z}^{\mathcal{V}_e}$ and concept $\mathbf{Z}^{\mathcal{V}_c}$. For each sampled subgraph $\mathcal{G}_e$

and $\mathcal{G}_c$, in order to find the best latent representation matrix, we apply Eqs. (4) and (9) to compute and update ϕ and θ in the inference model f_ϕ; the generative model g_θ. Finally, Eq. (16) will be used to optimize the whole problem.

Algorithm 1. The Joint GCN based DKT Framework

Input: sub-graphs $\mathcal{G}_e$ and $\mathcal{G}_c$; the inference model f_ϕ; the generative model g_θ; the interaction set $\mathcal{E}_s$
Output: latent representation matrix for the exercise $\mathbf{Z}^{\mathcal{V}_e}$ and concept $\mathbf{Z}^{\mathcal{V}_c}$

Initialize the inference model f_ϕ; the generative model g_θ
Initialize the latent representation matrix for the exercise $\mathbf{Z}^{\mathcal{A}_e}$ and concept attributes $\mathbf{Z}^{\mathcal{A}_c}$
for *each sampled subgraph* $\mathcal{G}_e$ *and* $\mathcal{G}_c$ **do**
  the mapping functions Ξ_c and Ξ_e as $\mathcal{G}_c \xrightarrow{\Xi_c} \mathbf{Z}_c^{\mathcal{V}}, \mathbf{Z}_c^{\mathcal{A}}$; $\mathcal{G}_e \xrightarrow{\Xi_e} \mathbf{Z}_e^{\mathcal{V}}, \mathbf{Z}_e^{\mathcal{A}}$
  for *epoch* $\leftarrow 1$ *to* T **do**
    apply Equation 4 to get $\{\sigma_{\mathcal{A}_{c,e}}, \mu_{\mathcal{A}_{c,e}}\}$
    update $\phi = [\mathbf{W}_{\mathcal{A}_c}, \mathbf{W}_{\mathcal{A}_e}, \mathbf{b}]$
  for *epoch* $\leftarrow 1$ *to* T **do**
    apply Eqation 9 to compute $\boldsymbol{\mu}_{\mathcal{E}_{\mathcal{A}}(i,j)}$ and $\boldsymbol{\sigma}^2_{\mathcal{E}_{\mathcal{A}}(i,j)}$
    update $\theta = [\theta_1, \theta_2]$
Compute the latent representation matrix for the exercise $\mathbf{Z}^{\mathcal{V}_e}$ and concept $\mathbf{Z}^{\mathcal{V}_c}$ using Equation 16.

4. Experiments

4.1. Datasets

Sufficient experiments have been conducted on four real-world KT datasets to demonstrate the efficacy of our framework for knowledge tracing as well as to provide a case study to demonstrate the potential interpretability of JKT for learning analytics.

- **ASSISTment 2009**[1] is offered by the online tutoring site ASSISTment and is commonly used for validation of KT problems. We performe our tests on the modified "concept-Builder" dataset. Students with no concepts or less than three records were discarded in preprocessing.
- **ASSISTment 2015**[2] collects 19,917 students' 708,631 interactions on 100 concepts. Although this dataset has more records than ASSISTment 2009, students' mean interaction records is lower due to the larger number of students.
- **ASSISTment Challenge**[3] (ASSISTment chall) is provided to the 2017 ASSISTments data mining competition. Concerning the mean number of records per student, it is richer as there are 686 students with 942,816 interactions and 102 concepts in the dataset.
- **STATICS 2011**[4] is a dataset of engineering statics course which contains 189,297 interactions on 1,223 concepts by 333 students. This dataset is the most dense of all the four datasets.

Table 2 shows the statistics of all datasets, $\#N_S$, $\#N_c$ and $\#N_\mathcal{E}$ represent the number of students, concepts, and "student-exercise" interactions, respectively.

4.2. Evaluation methods

We compare our proposed JKT with the following baseline methods.

[1] https://sites.google.com/site/assistmentsdata/home/assistment-2009–2010-data
[2] https://sites.google.com/site/assistmentsdata/home/2015-assistments-concept-builder-data
[3] https://sites.google.com/view/assistmentsdatamining/dataset
[4] https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId = 507

Table 2
Statistics for the datasets.

Datasets	Statistics		
	$\#N_S$	$\#N_c$	$\#N_\mathcal{E}$
ASSISTment 2009	4,151	110	325,637
ASSISTment 2015	19,917	100	708,631
ASSISTment chall	686	102	942,816
STATICS 2011	333	1,223	189,297

- **Bayesian Knowledge Tracing** [6] is a classical knowledge tracing model that traces the learning status of each student by using Hidden Markov Model to update and maintain a set of binary variables.
- **Deep Knowledge Tracing** [26] is a deep learning model. It uses the powerful learning capabilities of recurrent neural networks for sequential data to model the entire learning process of students. Its input is a one-hot embedding of the student-concept representations.
- **Dynamic Key-Value Memory Networks** [41], inspired by memory-enhanced neural networks, use a static matrix and a dynamic matrix to store all concepts and update the student's learning state respectively. However, it does not take into account the relationship of different exercises under the same concept.
- **SAKT** [24] uses self-attentive mechanisms to weight exercises that a student has previously worked on and adopts them to predict the subsequent exercises.
- **EKT** [19] is an extension of the EERNN model, which also traces students' knowledge acquisition of multiple concepts. Specifically, it models the relationships between foundational knowledge concepts to enhance the EERNN model.
- **SAINT+** [30] is an extension of SAINT, the first transformer-based knowledge tracing model with an encoder-decoder structure. A sequence of exercise embeddings is fed into the encoder, while the decoder can get the output of the exercise and its corresponding response embedding to compute the final output.

4.3. Ablation study

In this section, we perform some ablation studies to investigate the validity of two key components of our proposed model as follows, embeddings of exercise influence sub-graph and embeddings of concept influence sub-graph We set three comparison settings whose performance has been shown in Tables 3 and 4. The details of the three settings are as follows.

- JKT-c: We use the multi-hot embedding of the concept corresponding to each exercise instead of the embeddings of concept influence sub-graph as input to the Joint attributed knowledge tracing graph, which eliminates the influence of the embeddings of our learned concepts and only retains the "exercise-to-exercise" interactions under each concept to prove the validity of the embeddings of our learned concepts.
- JKT-e: We input the embeddings of concept influence sub-graph corresponding to each exercise directly into the Joint attributed knowledge tracing graph. This operation does not distinguish the difference between each exercise under a concept, but only retains the embeddings of concept influence sub-graph to prove the validity of the embedding of the exercise we learned.
- JKT: This operation preserves the complete model, while embedding the concept and exercise influence graph as input to the prediction layer to ensure maximum impact on the results.

4.4. Implement settings

For all the baseline models, we have referred to and reproduced the code published by the authors, and with sufficient adjustments, all parameters have been tuned to the best possible level to obtain the final results. For our model, we first find the optimal cases of the hyperparameters $\boldsymbol{\mu}_{\mathcal{A}_{c,e}}, \boldsymbol{\sigma}_{\mathcal{A}_{c,e}}$ of our prior distribution appearing in Eqs. (4) and (5) by multiple trials and set them to 1 in all experiments. we also train the model with the Adam optimizer for 300 iterations with a learning rate of 0.01. For our first- and second-layer neural networks, we respectively using a 64-dimensional hidden layer and a 32-dimensional latent variable. Our entire JKT model is based on a Pytorch implementation. The specific details of the procedure can be found in Algorithm 1.

4.5. Evaluation of effectiveness

As shown in Tables 3 and 4, the predictive performance of JKT as well as its variants are compared to the baseline methods. The *Area Under the curve (AUC)* and *Accuracy (ACC)* are used as the evaluation metrics. The problem of knowledge tracing is considered as a classification task, where exercise records with a prediction result of 1 or 0 indicate a positive or negative instance respectively. Therefore, two metrics are a common choice for this type of problem, namely prediction accuracy (ACC) and area under the ROC curve (AUC). In general, a larger value of AUC or ACC represents better prediction performance.

Table 3
Predictive performance. The best and the second best performing runs per metric per dataset are marked in boldface and underlined, respectively.

	ASSISTment 2009		ASSISTment 2015	
	AUC	*ACC*	*AUC*	*ACC*
BKT [6]	0.648	0.594	0.616	0.592
DKT [26]	0.74	0.708	0.701	0.68
DKVMN [41]	0.739	0.618	0.705	0.68
SAKT [24]	0.735	0.679	0.721	0.647
EKT [19]	0.754	0.702	0.737	<u>0.754</u>
SAINT+ [30]	<u>0.782</u>	<u>0.718</u>	<u>0.754</u>	0.741
JKT-c	0.717	0.671	0.718	0.681
JKT-e	0.714	0.694	0.747	0.684
JKT	**0.798**	**0.753**	**0.765**	**0.757**

Table 4
Predictive performance. The best and the second best performing runs per metric per dataset are marked in boldface and underlined, respectively.

	ASSISTment Chall		STATICS 2011	
	AUC	*ACC*	*AUC*	*ACC*
BKT [6]	0.562	0.555	0.719	0.698
DKT [26]	0.691	0.712	0.815	0.723
DKVMN [41]	0.689	0.614	0.814	0.722
SAKT [24]	0.701	0.657	0.803	0.797
EKT [19]	0.72	0.727	0.842	<u>0.819</u>
SAINT+ [30]	<u>0.734</u>	0.718	<u>0.853</u>	0.808
JKT-c	0.694	0.697	0.778	0.777
JKT-e	0.729	<u>0.733</u>	0.797	0.784
JKT	**0.745**	**0.737**	**0.856**	**0.832**

On the Assistment 2019 dataset, JKT outperformed the competing methods with an AUC and ACC of 0.788 and 0.732, compared to the second ranked SAINT+ with 0.782 and 0.718, respectively. For this dataset, JKT is very useful for removing the embeddings of concept and for this dataset, JKT-c and JKT-e with the effect of the embeddings of concept and exercise influence sub-graphs showed a significant improvement, with their AUC improving by about 11%, which demonstrates that the embeddings of these two sub-graphs have a positive and significant effect on the results. For this dataset, each individual exercise was from one concept or multiple concepts. The probability of a student correctly answering an exercise in this dataset was determined using Item Response Theory [29]. Thus, exercises belonging to the same concept are closely correlated in this dataset. Unlike other benchmarks, JKT learns the "concept-to-concept" relationship directly and thus performs better than other methods. In the datasets ASSISTment 2015 and ASSISTment Challenge, most of the methods did not perform as well as they did in the other two datasets. The main reason is that when the student set size and concept set size are imbalanced, JKT may produce excessive scale of student-concept representations with low distinction. This is a limitation of JKT to improve in the near future. Besides JKT, Saint+ has the worst performance, while the other models have some sorts of strengths and weaknesses at different situations. In the dataset STATICS2011, most of the methods showed the best prediction levels, and our method achieved a 1.5% performance improvement compared to EKT.

4.6. Demonstration of JKT's interpretability

In this subsection, we will have a discussion of the interpretability of JKT by introducing a visualized graph of exercise and concept embeddings and a meaningful case study.

Fig. 4 visualises the distribution of the embeddings of 100 selected concepts (red spots) and their corresponding exercises (blue spots) in ASSISTment 2009 using our proposed JKT model. This distribution shows that most concepts are independent each other, but some concepts are relatively close to each other. This shows the capability of our JKT to discover the implicit relationships of concepts and apply them to optimize the accuracy of knowledge tracing.

To show the details, we produce a case study by selecting five different concepts: "C1: Line graph", "C2: Probability", "C3: Linear equations", "C4: Vertical angles" and "C5: Inequality function" in Fig. 5.

The blue dotted line represents the level of mastery over a concept. The red, orange and green lines represent the mastery level of the three students over five concepts. We are not able to capture the mastery level of student 1 on "C4: vertical angles", as he has not done any exercises related to "C4: vertical angles". However, by using JKT's "concept-to-concept" relationship extraction, we could predict the student's mastery of C4. We can therefore obtain a complete mastery of each student on the selected concepts.

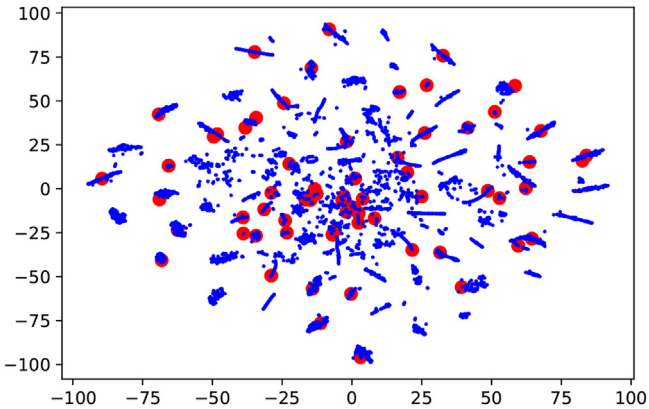

Fig. 4. A visualization of exercise and concept embeddings.

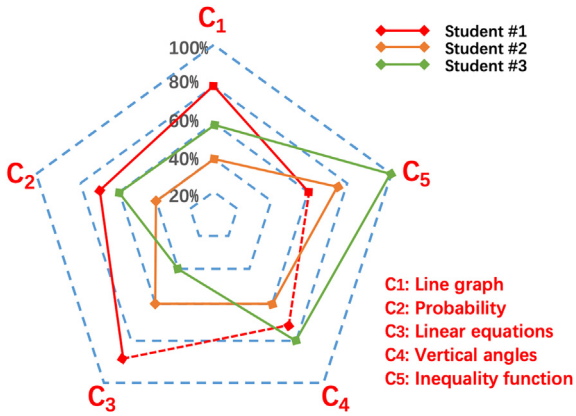

Fig. 5. A meaningful case study.

Based on this figure, we can also intuitively find the three students' inclinations towards different concepts. Overall, student 1 is a little more balanced, while student 3 is more biased and student 2 is relatively weak overall. This kind of overall mastery of a student on different concepts can be applied to many practical tasks, such as generating test papers, predicting grades and early intervention to prevent dropouts.

5. Conclusion and future work

In this work, we propose a Joint graph convolutional network based deep Knowledge Tracing (JKT), which adopts a novel inference-generating knowledge tracing framework. Jkt attempts to learn the conceptual potential embedding of exercises. By deep reconstruction of the original data and the original relations of "exercise-to-concept", JKT builds the multi-dimensional relationships of "*exercise-to-exercise*", and "*concept-to-concept*" and constructs their influence subgraphs, respectively. The framework maximises the preservation of the original structural information. And at the same time, through a process of inference and generation, JKT can learn the potential influences of exercises and concepts, which may lead to a more interpretable conceptual potential space. Extensive experimental results confirm the superior performance of our proposed model over the baseline methods.

For the future work, we will aim to optimise the efficiency of the framework and improve its interpretability. For example, we can introduce new mechanisms to identify potential connection paths between multiple exercises and to explore deep interpretability behind them. As this work focuses on the first tier of "*exercise-to-exercise*", and "*concept-to-concept*" relationship, we leave this potential improvement for future work.

CRediT authorship contribution statement

Xiangyu Song: Conceptualization, Methodology, Data curation, Writing - original draft preparation. **Jianxin Li:** Conceptualization, Methodology, Writing - review & editing. **Yifu Tang:** Data curation, Software. **Taige Zhao:** Data curation, Software. **Yunliang Chen:** Supervision. **Ziyu Guan:** Writing - review & editing, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] G. Abdelrahman, Q. Wang, Knowledge tracing with sequential key-value memory networks, in: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2019, pp. 175–184.
[2] J.R. Anderson, C.F. Boyle, B.J. Reiser, Intelligent tutoring systems, Science 228 (4698) (1985) 456–462.
[3] L.E. Baum, T. Petrie, Statistical inference for probabilistic functions of finite state markov chains, Ann. Math. Stat. 37 (6) (1966) 1554–1563.
[4] J. Bruna, W. Zaremba, A. Szlam, Y. LeCun, Spectral networks and locally connected networks on graphs, in: 2nd International Conference on Learning Representations, ICLR, 2014.
[5] S. Cao, W. Lu, Q. Xu, Grarep: Learning graph representations with global structural information, in: Proceedings of the 24th ACM International on Conference on Information and Knowledge Management, 2015, pp. 891–900.
[6] A.T. Corbett, J.R. Anderson, Knowledge tracing: Modelling the acquisition of procedural knowledge, User Model. User Adapt. Interact. 4 (4) (1995) 253–278.
[7] C. de Bodt, D. Mulders, M. Verleysen, J.A. Lee, Nonlinear dimensionality reduction with missing data using parametric multiple imputations, IEEE Trans. Neural Networks Learn. Syst. 30 (4) (2019) 1166–1179.
[8] C.B. Do, S. Batzoglou, What is the expectation maximization algorithm?, Nat Biotechnol. 26 (8) (2008) 897–899.
[9] A. Ghosh, N. Heffernan, A.S. Lan, Context-aware attentive knowledge tracing, in: KDD'20: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020.
[10] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 855–864.
[11] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems, 2017, pp. 1024–1034. .
[12] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[13] X. Huang, J. Li, X. Hu, Accelerated attributed network embedding, in: Proceedings of the 2017 SIAM International Conference on Data Mining, SIAM, 2017, pp. 633–641.
[14] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015. .
[15] D.P. Kingma, M. Welling, Auto-encoding variational bayes, in: 2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14–16, 2014, Conference Track Proceedings, 2014. .
[16] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings, OpenReview.net, 2017. .
[17] J. Li, H. Dani, X. Hu, J. Tang, Y. Chang, H. Liu, Attributed network embedding for learning in a dynamic environment, in: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM 2017, Singapore, November 06–10, 2017, ACM, 2017, pp. 387–396. .
[18] Y. Liang, C. Huang, X. Bao, K. Xu, Sequential dynamic event recommendation in event-based social networks: An upper confidence bound approach, Inf. Sci. 542 (2021) 1–23.
[19] Q. Liu, Z. Huang, Y. Yin, E. Chen, H. Xiong, Y. Su, G. Hu, Ekt: Exercise-aware knowledge tracing for student performance prediction, IEEE Trans. Knowl. Data Eng. 33 (1) (2019) 100–115.
[20] S. Minn, M.C. Desmarais, F. Zhu, J. Xiao, J. Wang, Dynamic student classiffication on memory networks for knowledge tracing, in: Advances in Knowledge Discovery and Data Mining - 23rd Pacific-Asia Conference, PAKDD 2019, Macau, China, April 14–17, 2019, Proceedings, Part II, Vol. 11440 of Lecture Notes in Computer Science, Springer, 2019, pp. 163–174. .
[21] S. Montero, A. Arora, S. Kelly, B. Milne, M. Mozer, Does deep knowledge tracing model interactions among skills?, in: Proceedings of the 11th International Conference on Educational Data Mining, EDM 2018, International Educational Data Mining Society (IEDMS), 2018
[22] A.B. Morton, I.M. Mareels, An efficient brute-force solution to the network reconfiguration problem, IEEE Trans. Power Delivery 15 (3) (2000) 996–1000.
[23] H. Nakagawa, Y. Iwasawa, Y. Matsuo, Graph-based knowledge tracing: Modeling student proficiency using graph neural network, in: 2019 IEEE/WIC/ACM International Conference on Web Intelligence, WI 2019, Thessaloniki, Greece, October 14–17, 2019, ACM, 2019, pp. 156–163. .
[24] S. Pandey, G. Karypis, A self attentive model for knowledge tracing, in: Proceedings of the 12th International Conference on Educational Data Mining, EDM 2019, International Educational Data Mining Society (IEDMS), 2019.
[25] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: online learning of social representations, in: The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, New York, NY, USA - August 24–27, 2014, ACM, 2014, pp. 701–710. .
[26] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L.J. Guibas, J. Sohl-Dickstein, Deep knowledge tracing, in: Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7–12, 2015, Montreal, Quebec, Canada, 2015, pp. 505–513. .
[27] J. Qiu, Y. Dong, H. Ma, J. Li, K. Wang, J. Tang, Network embedding as matrix factorization: Unifying deepwalk, line, pte, and node2vec, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, WSDM 2018, Marina Del Rey, CA, USA, February 5–9, 2018, ACM, 2018, pp. 459–467. .
[28] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, Nature 323 (6088) (1986) 533–536.
[29] J. Self, Theoretical foundations for intelligent tutoring systems, J. Artif. Intell. Edu. 1 (4) (1990) 3–14.
[30] D. Shin, Y. Shim, H. Yu, S. Lee, B. Kim, Y. Choi, SAINT+: integrating temporal features for ednet correctness prediction, in: LAK'21: 11th International Learning Analytics and Knowledge Conference, Irvine, CA, USA, April 12–16, 2021, ACM, 2021, pp. 490–496. .
[31] Y. Su, Q. Liu, Q. Liu, Z. Huang, Y. Yin, E. Chen, C.H.Q. Ding, S. Wei, G. Hu, Exercise-enhanced sequential modeling for student performance prediction, in: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, AAAI Press, 2018, pp. 2435–2443.
[32] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei, LINE: large-scale information network embedding, in: Proceedings of the 24th International Conference on World Wide Web, WWW 2015, Florence, Italy, May 18–22, 2015, ACM, 2015, pp. 1067–1077. .
[33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA, 2017, pp. 5998–6008. .
[34] C. Wang, B. Samari, K. Siddiqi, Local spectral graph convolution for point set feature learning, in: Computer Vision - ECCV 2018–15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part IV, Vol. 11208 of Lecture Notes in Computer Science, Springer, 2018, pp. 56–71. .
[35] D. Wang, P. Cui, W. Zhu, Structural deep network embedding, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 1225–1234.

[36] T. Wang, F. Ma, J. Gao, Deep hierarchical knowledge tracing, in: Proceedings of the 12th International Conference on Educational Data Mining, EDM 2019, International Educational Data Mining Society (IEDMS), 2019.
[37] Y. Wang, Y. Yuan, G. Wang, Y. Ma, Graph cells: Top-k structural-textual aggregated query over information networks, Inf. Sci. 547 (2021) 354–366.
[38] C. Yang, Z. Liu, D. Zhao, M. Sun, E.Y. Chang, Network representation learning with rich text information, in: Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25–31, 2015, AAAI Press, 2015, pp. 2111–2117. .
[39] J.W. Young, A first order approximation to the optimum checkpoint interval, Commun. ACM 17 (9) (1974) 530–531.
[40] Q. Zeng, M. Zhong, Y. Zhu, T. Qian, J. Li, Business location planning based on a novel geo-social influence diffusion model, Inf. Sci. 559 (2021) 61–74.
[41] J. Zhang, X. Shi, I. King, D. Yeung, Dynamic key-value memory networks for knowledge tracing, in: Proceedings of the 26th International Conference on World Wide Web, WWW 2017, Perth, Australia, April 3–7, 2017, ACM, 2017, pp. 765–774. .
[42] M. Zinkevich, Online convex programming and generalized infinitesimal gradient ascent, in: Machine Learning, Proceedings of the Twentieth International Conference (ICML 2003), August 21–24, 2003, Washington, DC, USA, AAAI Press, 2003, pp. 928–936. .